

调试

◆ 动态调试

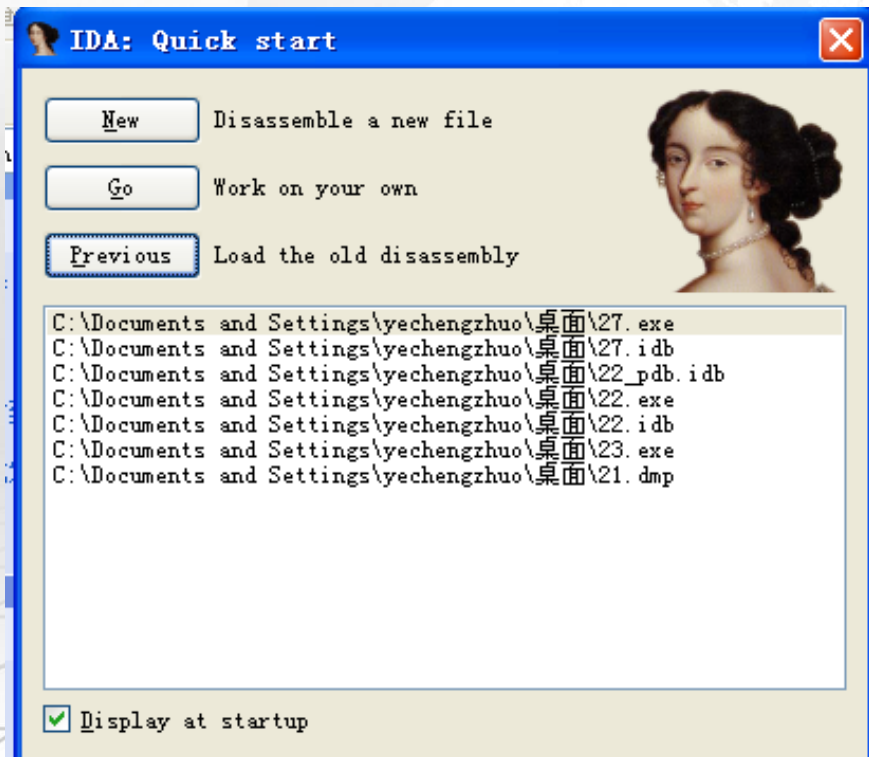
OllyDbg

◆ 静态调试

IDApr

一、1 IDA入门

1.1 IDA的启动和加载文件



打开IDA后会后三种选择：

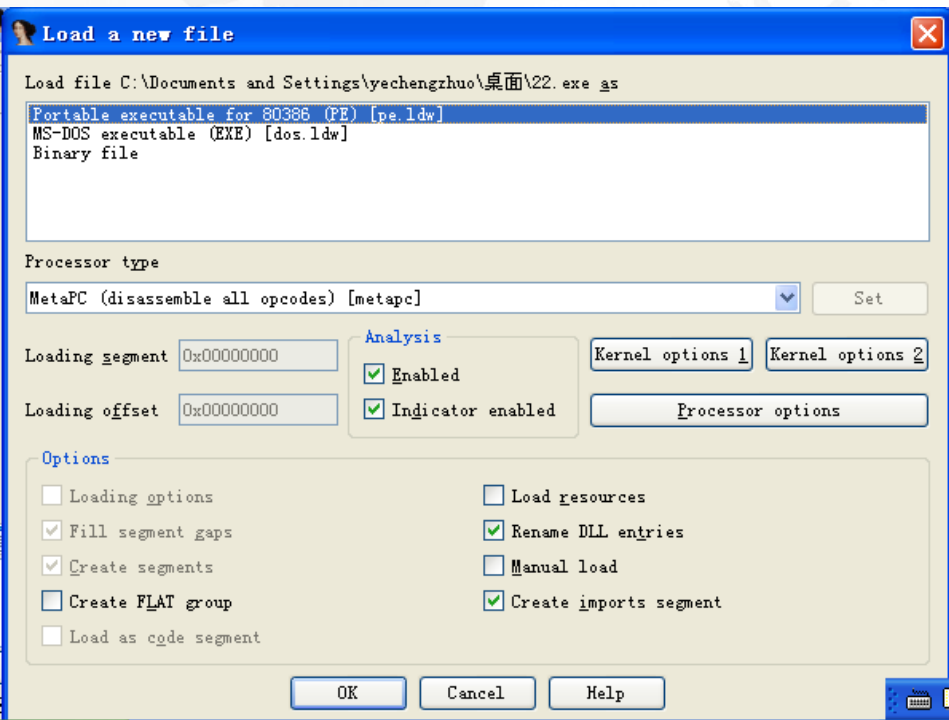
New（新建）：将启动一个标准的File Open对话框来选择将要分析的文件；

Go（运行）：将终止加载过程，使IDA打开一个空包的工作区，可通过将一个二进制文件直接拖放到IDA工作区或使用File菜单中的某个选项打开文件；

Previous（上一个）：可打开其下“最近用过的文件”列表中的一个。

一、1 IDA入门

1.1 IDA的启动和加载文件



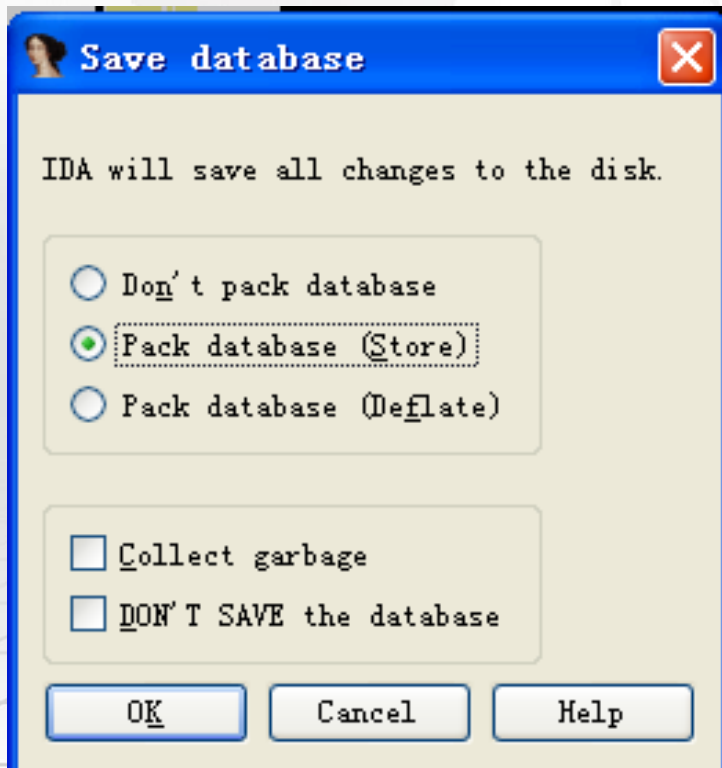
➤ IDA文件加载：
PE加载器和MS-DOS EXE加载器能够识别选定的文件；Binary File加载无法识别的文件，它提供了最低级的文件加载方法。

- 处理器
- 基地址与偏移
- 可选项

加载资源
手工加载
窗表导入表段

一、1 IDA入门

1.2 IDA数据库文件



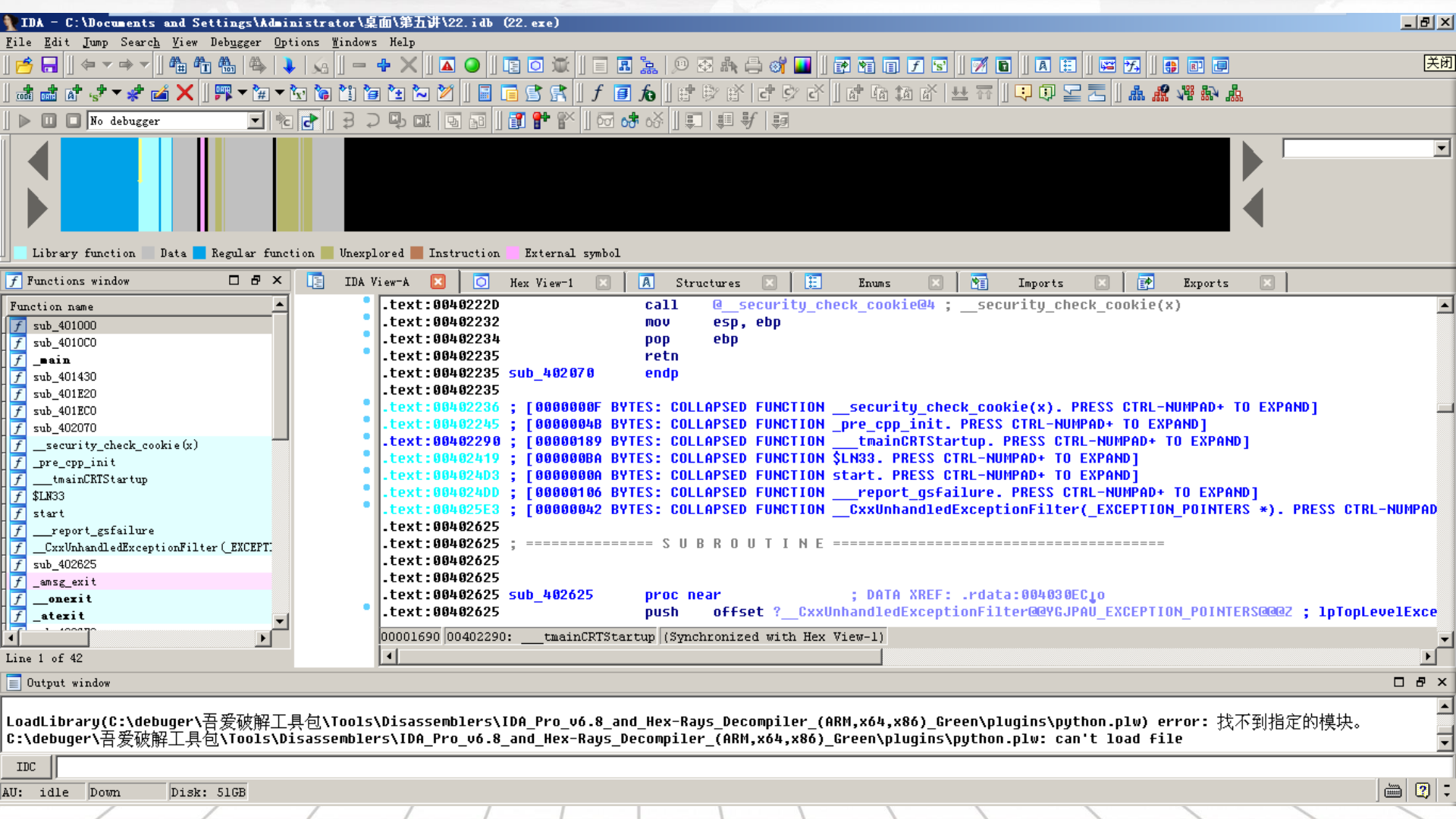
无论是关闭IDA还是切换到另一个数据库，IDA都会显示一个Save database对话框：

Don't pack database：不创建IDB文件，只刷新对4个数据库组件文件所做的更改；

Pack database (Store)：将4个数据库组件文件保存到一个IDB文件中，但不会压缩；

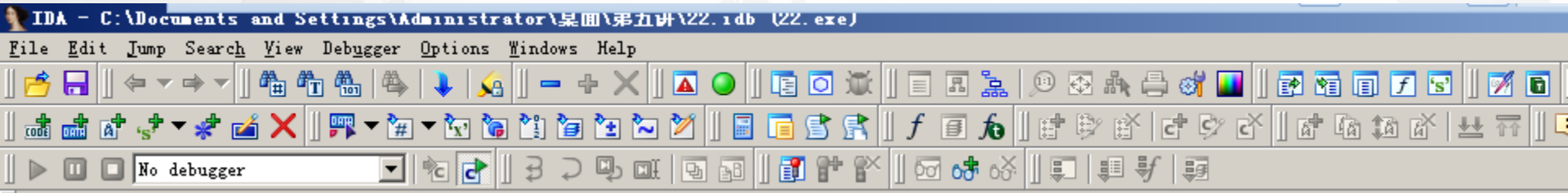
Pack database (Deflate)：等同于Store选项，但是会压缩到IDB归档文件中；

DON'T SAVE the database：删除4个数据库组件文件，保留现有的未经修改的IDB文件。



一、1 IDA入门

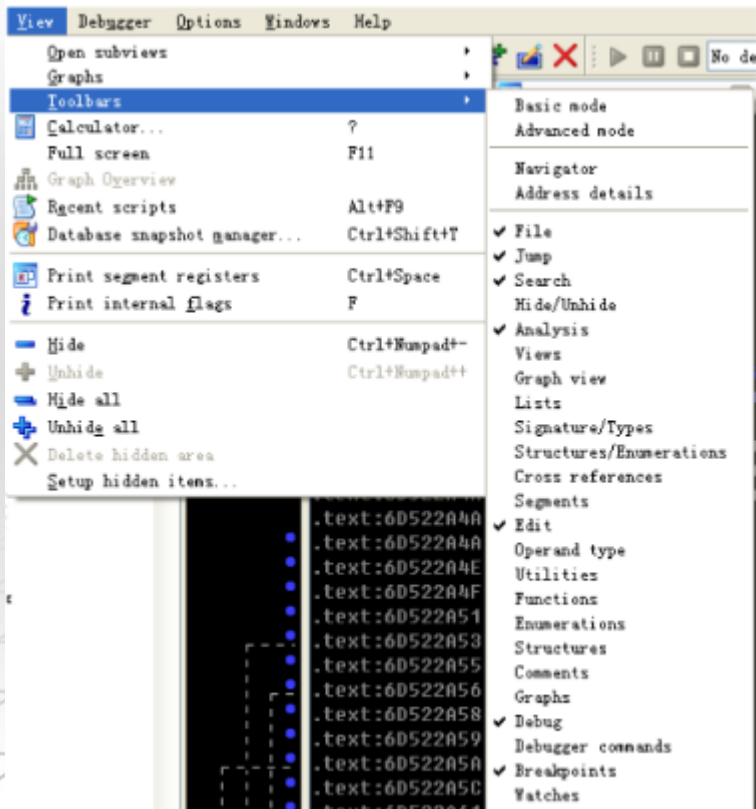
1.3 IDA桌面简介



工具栏区域包含与IDA的常用操作对应的工具，在“View” -> “ToolBars”下有两个子菜单，“Base IC mode”和“Advance mode”，也可以通过右键来设置工具栏

一、1 IDA入门

1.3 IDA桌面简介



■ 工具栏

- 视图
- 结构体&枚举
- 区段
- 函数
- 交叉引用
- 调试与断点

一、1 IDA入门

菜单栏-跳转

■ 跳转到[]

- 地址
- 名字
- 函数
- 段

■ 标签

- 添加/删除标签
- 跳转到标签

	Jump to operand	Enter
	Jump in a new window	Alt+Enter
	Jump to previous position	Esc
	Jump to next position	Ctrl+Enter
	Empty navigation stack	
	Jump to pseudocode	Tab
	Jump to address...	G
	Jump by name...	Ctrl+L
	Jump to function...	Ctrl+P
	Jump to segment...	Ctrl+S
	Jump to segment register...	Ctrl+G
	Jump to problem...	Ctrl+Q
	List cross references to...	Ctrl+X
	List cross references from...	Ctrl+J
	Jump to xref to operand...	X
	Jump to entry point...	Ctrl+E
	Jump to file offset...	
	Mark position...	Alt+M
	Jump to marked position...	Ctrl+M
	Clear mark...	

一、1 IDA入门

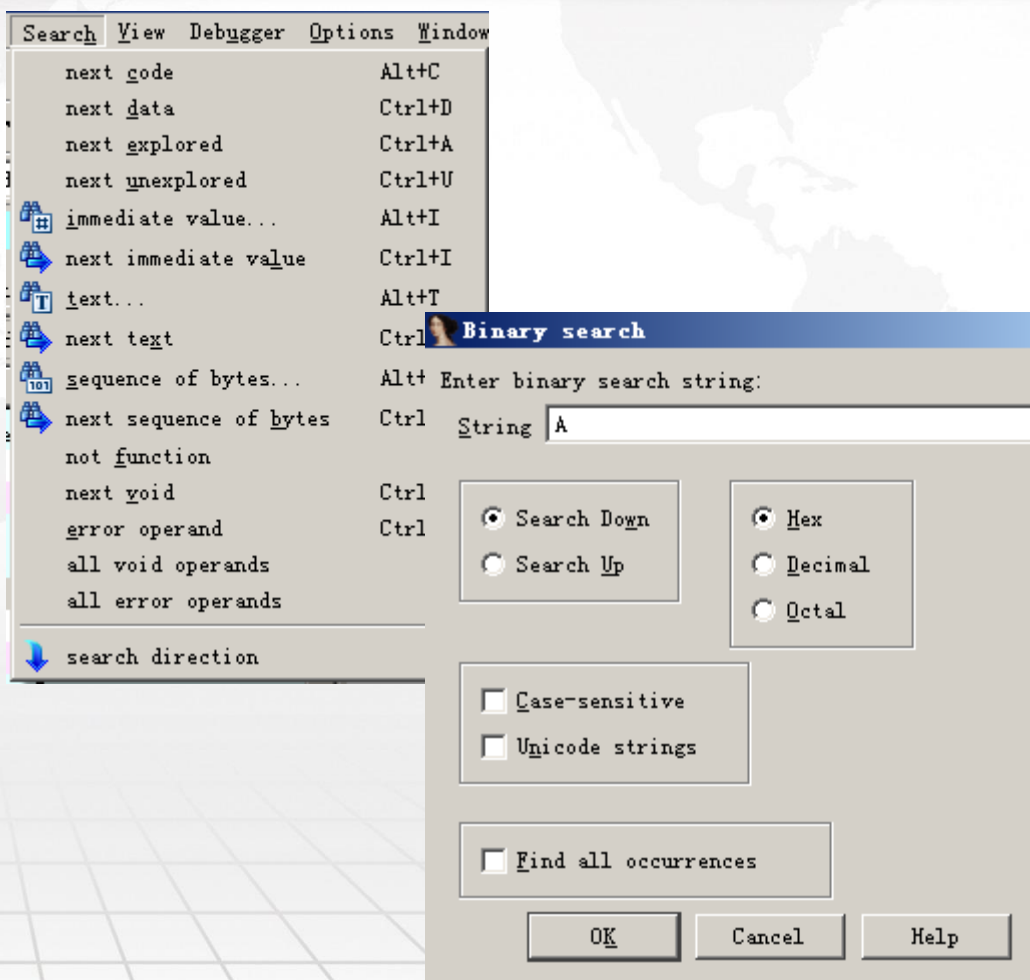
菜单栏-搜索

■ 收索

- 字符串
- 二进制

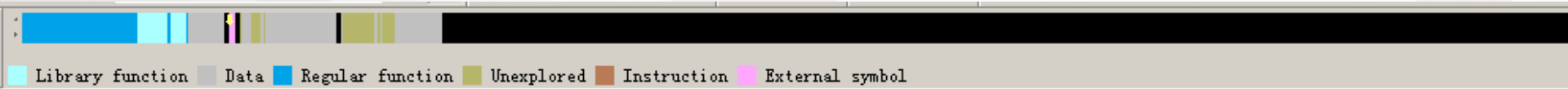
■ 技巧

- 搜索提示字符串
- 搜索特征字
- 注意是否设置为全局搜索



一、1 IDA入门

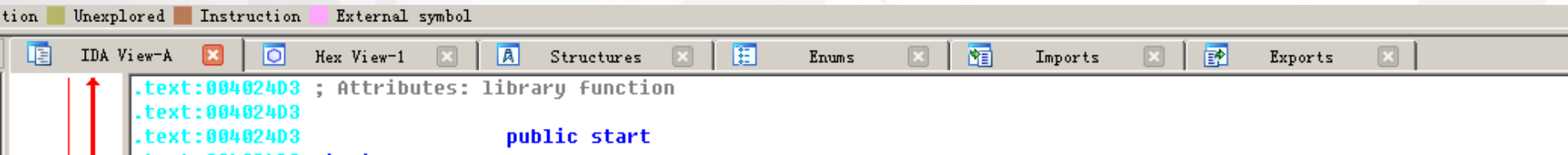
1.3 IDA桌面简介



导航带是被加载文件地址空间的线性视图，它会呈现二进制文件的整个地址范围，通过右击导航带内任何位置，并选择一个可用的缩放选项，放大或缩小显示的地址范围。

一、1 IDA入门

逆向工作区



View->open subviews打开



IDA 数据显示窗口

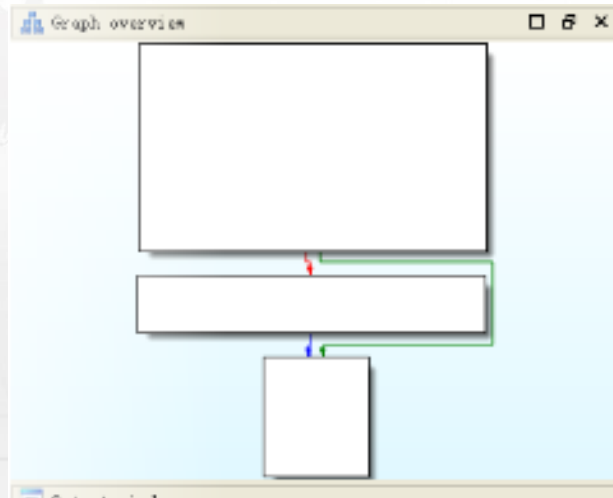


2 IDA数据显示窗口

2.1 IDA主要数据显示窗口：反汇编窗口（图形视图）

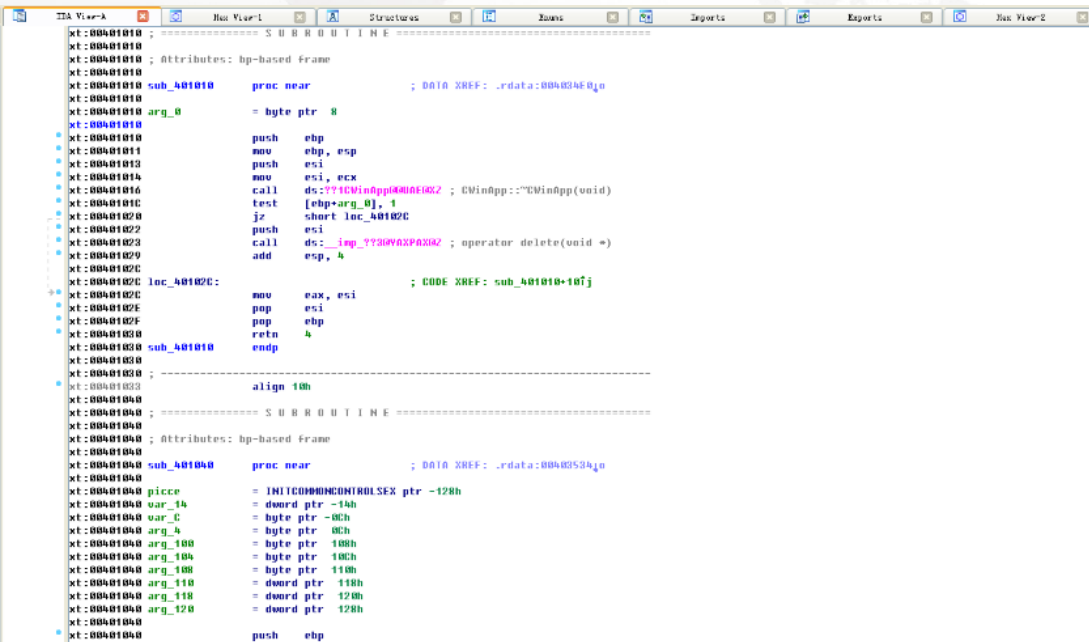


IDA使用不同的彩色箭头区分函数块之间各种类型的流。可使用CTRL+鼠标滑轮调整图形大小，可通过图形概况窗口迅速定位。



2 IDA数据显示窗口

2.1 IDA主要数据显示窗口：反汇编窗口（文本视图）



```
xt:00401010 ; ===== SUBROUTINE =====
xt:00401010 ; Attributes: bp-based frame
xt:00401010
xt:00401010 sub_401010 proc near ; DATA XREF: .rdata:00403040n
xt:00401010 arg_0 = byte ptr 8
xt:00401010
xt:00401010 push ebp
xt:00401011 mov ebp, esp
xt:00401013 push esi
xt:00401014 mov esi, ecx
xt:00401016 call ds:??1CWinApp@00001002 ; CWinApp::CWinApp(void)
xt:0040101C test [ebp+arg_0], 1
xt:00401020 jz short loc_40102C
xt:00401022 push esi
xt:00401023 call ds:imp_??309903P0302 ; operator delete(void *)
xt:00401029 add esp, 4
xt:0040102E
xt:0040102E loc_40102C: ; CODE XREF: sub_401010+10fj
xt:0040102E mov eax, esi
xt:0040102F pop esi
xt:0040102F pop ebp
xt:00401030 ret 4
xt:00401030 sub_401010 endp
xt:00401033 ; -----
xt:00401033 align 10h
xt:00401040 ; ===== SUBROUTINE =====
xt:00401040 ; Attributes: bp-based frame
xt:00401040
xt:00401040 sub_401040 proc near ; DATA XREF: .rdata:00403040n
xt:00401040 arg_0 = INITCOMMONCONTROLSEX ptr -128h
xt:00401040 var_14 = dword ptr -14h
xt:00401040 var_0 = byte ptr -0Ch
xt:00401040 arg_4 = byte ptr 0Ch
xt:00401040 arg_100 = byte ptr 100h
xt:00401040 arg_104 = byte ptr 10Ch
xt:00401040 arg_108 = byte ptr 110h
xt:00401040 arg_110 = dword ptr 110h
xt:00401040 arg_118 = dword ptr 120h
xt:00401040 arg_120 = dword ptr 128h
xt:00401040
xt:00401040 push ebp
```

文本窗口呈现一个程序的完整反汇编代码清单（图形模式下一次只能显示一个函数），可通过“空格键”切换对应函数的图形和文本视图。

2 IDA数据显示窗口

2.1 IDA主要数据显示窗口：函数窗口

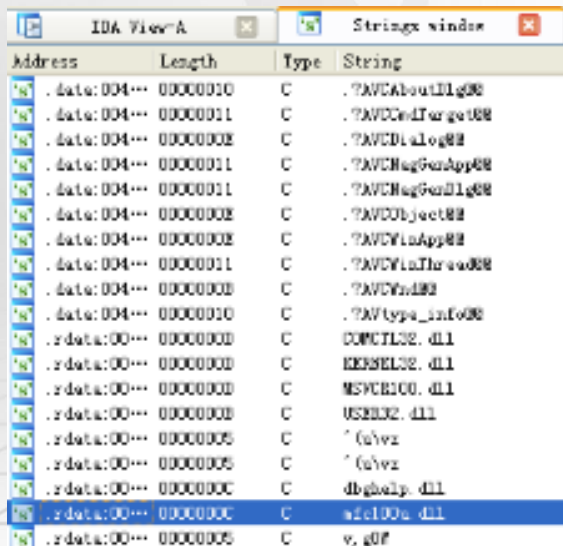
函数窗口提供所有函数名，可以通过函数窗口迅速定位到对应的文本视图或图形视图。

Functions window											
Function name	Segment	Start	Length	Locals	Arguments	B	F	L	S	D	T
xub_401000	.text	00401000	00000008			?	-	-	-	-	-
xub_401010	.text	00401010	00000025	00000008	00000001	?	-	-	-	B	-
xub_401040	.text	00401040	000000FF	00000138	00000124	?	-	-	-	B	-
CWinApp::GetApplicationRecoveryPingInt...	.text	00401170	00000006			?	-	L	-	-	T
xub_401180	.text	00401180	00000003			?	-	-	-	-	-
CWinApp::RestoreAutoSavedFilesAtRestart...	.text	00401190	0000000A			?	-	L	-	-	T
CWinApp::ReopenPreviousFilesAtRestart(...	.text	004011A0	0000000A			?	-	L	-	-	T
CWinApp::SupportsAutoSaveAtInterval (voi...	.text	004011B0	0000000A			?	-	L	-	-	T
CWinApp::SupportsAutoSaveAtRestart (void)	.text	004011C0	0000000A			?	-	L	-	-	T
CWinApp::SupportsApplicationRecovery (voi...	.text	004011D0	0000000A			?	-	L	-	-	T
CWinApp::SupportsRestartManager (void)	.text	004011E0	0000000A			?	-	L	-	-	T
nullxub_1	.text	00401170	00000003	00000000	00000000	?	-	-	-	-	-
nullxub_2	.text	00401200	00000001			?	-	-	-	-	-
xub_401210	.text	00401210	00000023	00000008	00000001	?	-	-	-	B	-
xub_401250	.text	00401250	00000006			?	-	-	-	-	-
xub_401260	.text	00401260	00000129	00000018	00000004	?	-	-	-	B	-
xub_401300	.text	00401300	00000049	00000008	00000001	?	-	-	-	B	-
xub_401380	.text	00401380	00000052	00000010	00000004	?	-	-	-	B	-
xub_401450	.text	00401450	00000006			?	-	-	-	-	-
xub_401460	.text	00401460	00000107	00000024	00000000	?	-	-	-	B	-
CWFCVisualManager::GetTasksPaneGroupCa...	.text	00401700	00000007			?	-	L	-	-	T
xub_401E40	.text	00401E40	000000F5	00000008	00000004	?	-	-	-	B	-
xub_401E40	.text	00401E40	00000016	00000004	00000008	?	-	-	-	B	-
xub_401E00	.text	00401E00	00000000			?	-	-	-	-	-
xub_401E00	.text	00401E00	00000000			?	-	-	-	-	-
xub_401E00	.text	00401E00	0000007C			?	-	-	-	-	-
xub_401F60	.text	00401F60	00000080	00000004	00000004	?	-	-	-	B	-
xub_401FF0	.text	00401FF0	0000001E	00000004	00000000	?	-	-	-	-	-
xub_402010	.text	00402010	0000007C	00000008	00000000	?	-	-	-	-	-
xub_402030	.text	00402030	00000041	00000008	00000000	?	-	-	-	-	-
xub_4020E0	.text	004020E0	00000055	0000000C	00000008	?	-	-	-	B	T
operator delete (void *)	.text	00402136	00000006			?	-	L	-	-	T

2 IDA数据显示窗口

2.1 IDA主要数 据显示窗口： Strings窗口

Strings窗口是IDA中内置的，显示的是从二进制文件中提取出的一组字符串，以及每个字符串所在的地址，可通过shift+F12迅速打开。

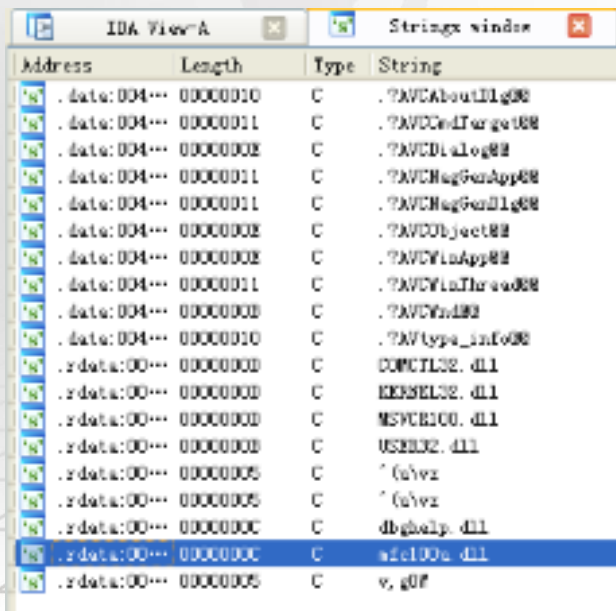


Address	Length	Type	String
.data:004...	00000010	C	.TAVCAboutDlg00
.data:004...	00000011	C	.TAVCCedTarget00
.data:004...	00000002	C	.TAVCDialog00
.data:004...	00000011	C	.TAVCHagGenApp00
.data:004...	00000011	C	.TAVCHagGenDlg00
.data:004...	00000002	C	.TAVCObject00
.data:004...	00000002	C	.TAVCWinApp00
.data:004...	00000011	C	.TAVCWinThread00
.data:004...	00000002	C	.TAVCWin00
.data:004...	00000010	C	.TAVType_info00
.rdata:00...	00000000	C	COMCTL32.dll
.rdata:00...	00000000	C	EXESEL32.dll
.rdata:00...	00000000	C	MSVCR100.dll
.rdata:00...	00000000	C	USER32.dll
.rdata:00...	00000005	C	"C:\vcr
.rdata:00...	00000005	C	"C:\vcr
.rdata:00...	0000000C	C	dbghelp.dll
.rdata:00...	0000000C	C	advapi32.dll
.rdata:00...	00000005	C	v, g0f

2 IDA数据显示窗口

2.2 IDA次要数据显示窗

□：Names窗口



Address	Length	Type	String
.data:004...	00000010	C	.?AVCAboutDlg@8
.data:004...	00000011	C	.?AVCCEditTarget@8
.data:004...	0000000E	C	.?AVCDialog@8
.data:004...	00000011	C	.?AVCHagGenApp@8
.data:004...	00000011	C	.?AVCHagGenDlg@8
.data:004...	0000000E	C	.?AVCObject@8
.data:004...	0000000E	C	.?AVCVinApp@8
.data:004...	00000011	C	.?AVCVinThread@8
.data:004...	0000000B	C	.?AVCWin@8
.data:004...	00000010	C	.?AVType_info@8
.rdata:00...	0000000B	C	COMCTL32.dll
.rdata:00...	0000000B	C	USER32.dll
.rdata:00...	0000000B	C	MSVCRT00.dll
.rdata:00...	0000000B	C	USER32.dll
.rdata:00...	00000005	C	^?u?ez
.rdata:00...	00000005	C	^?u?ez
.rdata:00...	0000000C	C	dbgshelp.dll
.rdata:00...	0000000C	C	wic100a.dll
.rdata:00...	00000005	C	v, g0#

Names窗口列举了一个二进制文件所有全局名称，即一个程序虚拟地址的符号描述，Names窗口中显示的名称采用了颜色和字母编码。

F：常规函数；

L：库函数（主体将在反汇编代码清单中显示）；

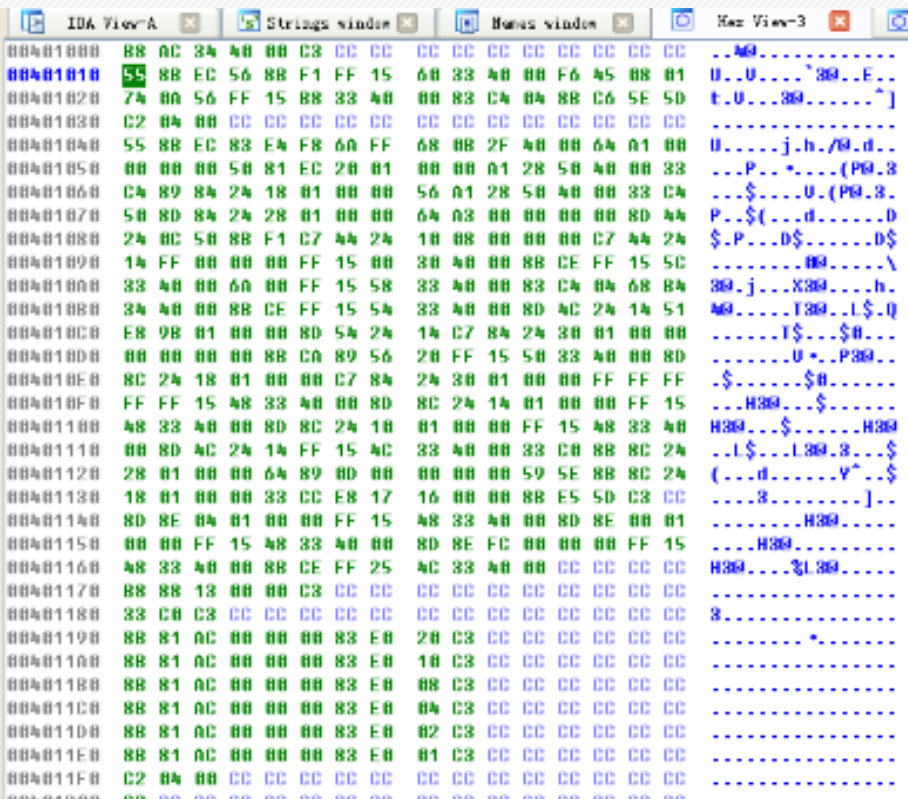
I：共享库导入的函数（导入名称没有代码）；

D：数据；

A：字符串数据。

2 IDA数据显示窗口

2.2 IDA次要数据显示窗口： 十六进制窗口 (Hex View)



此窗口显示程序内容和列表的标准十六进制代码，每行显示16个字节，以及对应的ASCII字符。Hex View窗口与反汇编窗口是同步的汇编窗口中的位置在Hex View中会高亮显示。

■ 布局

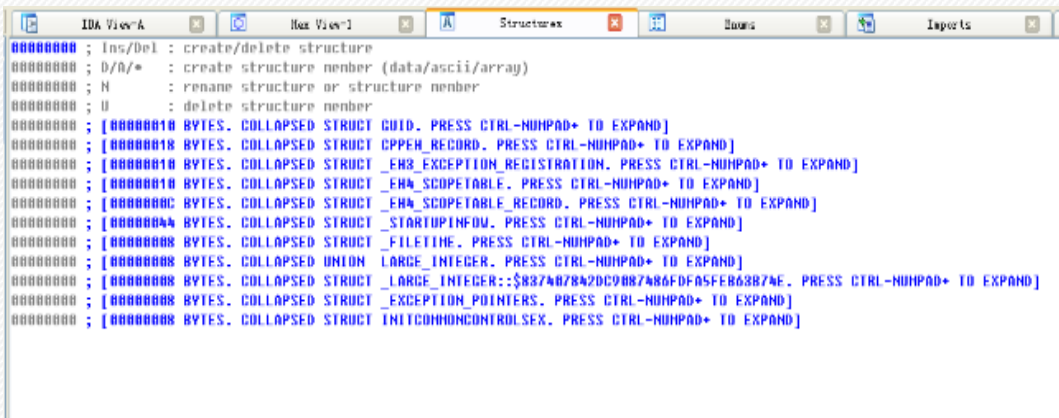
- 左侧是地址
- 中间是16进制
- 右侧是对应得ASCII

■ 操作

- 跳转到指定地址 g
- 对指定地址得字节进行编辑

2 IDA数据显示窗口

2.2 IDA次要数据显示窗口： 结构体窗口



```
00000000 ; Ins/Del : create/delete structure
00000000 ; D/A/* : create structure member (data/ascii/array)
00000000 ; R : rename structure or structure member
00000000 ; U : delete structure member
00000000 ; [00000010 BYTES, COLLAPSED STRUCT GUID, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000010 BYTES, COLLAPSED STRUCT CPPEN_RECORD, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000010 BYTES, COLLAPSED STRUCT _EXCEPTION_REGISTRATION, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000010 BYTES, COLLAPSED STRUCT _EXN_SCOPEABLE, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [0000000C BYTES, COLLAPSED STRUCT _EXN_SCOPEABLE_RECORD, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000004 BYTES, COLLAPSED STRUCT _STARTUPINFOW, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000008 BYTES, COLLAPSED STRUCT _FILETIME, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000008 BYTES, COLLAPSED UNION LARGE_INTEGER, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000008 BYTES, COLLAPSED STRUCT _LARGE_INTEGER::{$837407842DC9087486F0F05F063874E, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000008 BYTES, COLLAPSED STRUCT _EXCEPTION_POINTERS, PRESS CTRL-NUMPAD+ TO EXPAND]
00000000 ; [00000008 BYTES, COLLAPSED STRUCT INITCOMMONCONTROLSEX, PRESS CTRL-NUMPAD+ TO EXPAND]
```

此窗口用于显示IDA决定在一个二进制文件中使用的任何复杂的数据结构的布局。

■ 说明

- 所有结构体的声明与定义都在这里

■ 操作

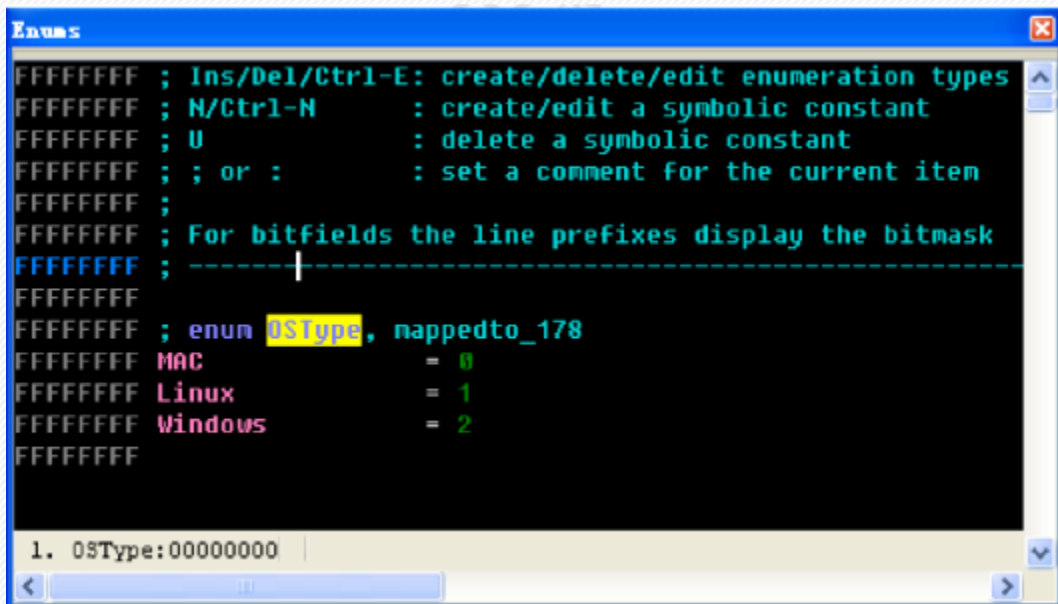
- 添加/删除结构体
- 重命名结构体
- 添加/删除结构体成员
- 重命名结构体成员
- 结构体排序

■ 创建结构体变量

- 在代码段或数据段

2 IDA数据显示窗口

2.2 IDA次要数据显示窗口： 枚举窗口



The screenshot shows the 'Enums' window in IDA. The main area is a black console-style window with green text. It contains instructions for creating and editing enumeration types, followed by a definition of an enumeration named 'OSType' with values for 'MAC', 'Linux', and 'Windows'. The 'OSType' value is highlighted in yellow. At the bottom, there is a status bar showing '1. OSType:00000000'.

```
FFFFFFFF ; Ins/Del/Ctrl-E: create/delete/edit enumeration types
FFFFFFFF ; N/Ctrl-N      : create/edit a symbolic constant
FFFFFFFF ; U            : delete a symbolic constant
FFFFFFFF ; ; or :       : set a comment for the current item
FFFFFFFF ;
FFFFFFFF ; For bitfields the line prefixes display the bitmask
FFFFFFFF ; -----
FFFFFFFF
FFFFFFFF ; enum OSType, nappedto_178
FFFFFFFF MAC          = 0
FFFFFFFF Linux        = 1
FFFFFFFF Windows      = 2
FFFFFFFF
```

1. OSType:00000000

此窗口类似结构体窗口，如果IDA检测到标准枚举数据类型，它将在枚举窗口中列出该数据类型。

■说明

- 这里只是定义

■操作

- 添加/删除/编辑枚举类型
- 添加/删除/编辑枚举条目

■创建

- 找到常量
- 重新设置为枚举类型

3 反汇编导航

导入表

Imports			
Address	Ordinal	Name	Library
0040B020		InterlockedIncrement	KERNEL32
0040B024		GetProcAddress	KERNEL32
0040B028		lstrcpyW	KERNEL32
0040B02C		InterlockedExchange	KERNEL32
0040B030		LocalFree	KERNEL32
0040B034		GetModuleHandleW	KERNEL32
0040B038		LoadLibraryW	KERNEL32
0040B03C		GetLastError	KERNEL32
0040B040		DeactivateActCtx	KERNEL32
0040B044		InterlockedDecrement	KERNEL32
0040B048		Sleep	KERNEL32
0040B04C		IsDebuggerPresent	KERNEL32
0040B050		UnhandledExceptionFilter	KERNEL32
0040B054		GetCurrentProcess	KERNEL32
0040B058		TerminateProcess	KERNEL32
0040B05C		GetSystemTimeAsFileTime	KERNEL32
0040B060		GetCurrentProcessId	KERNEL32
0040B064		GetCurrentThreadId	KERNEL32
0040B068		GetTickCount	KERNEL32
0040B06C		QueryPerformanceCounter	KERNEL32
0040B070		SetUnhandledExceptionFilter	KERNEL32

Line 17 of 659

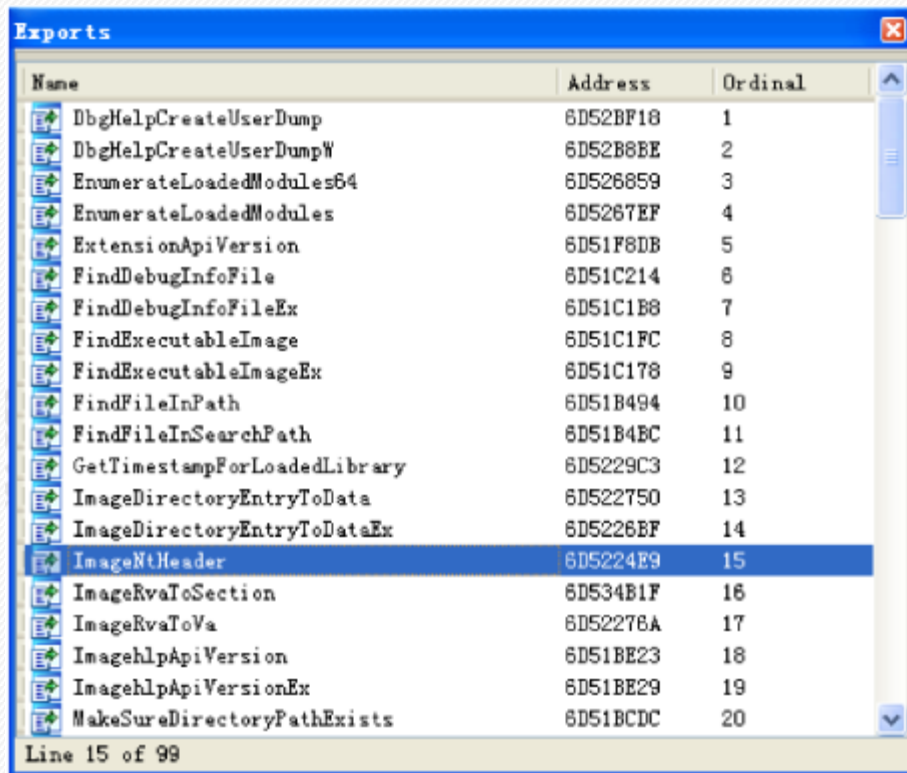
■ 布局

- 左侧是地址
- 中间是引用到的函数的名字
- 右侧是函数所在的DLL

■ 使用

- 选中感兴趣的API双击跳过去
- 通过交叉引用，查看上下文

导出表



Name	Address	Ordinal
DbgHelpCreateUserDump	6D52BF18	1
DbgHelpCreateUserDumpW	6D52B8BE	2
EnumerateLoadedModules64	6D526859	3
EnumerateLoadedModules	6D5267EF	4
ExtensionApiVersion	6D51F8DB	5
FindDebugInfoFile	6D51C214	6
FindDebugInfoFileEx	6D51C1B8	7
FindExecutableImage	6D51C1FC	8
FindExecutableImageEx	6D51C178	9
FindFileInPath	6D51B494	10
FindFileInSearchPath	6D51B4BC	11
GetTimestampForLoadedLibrary	6D5229C3	12
ImageDirectoryEntryToData	6D522750	13
ImageDirectoryEntryToDataEx	6D5226BF	14
ImageNtHeader	6D5224F9	15
ImageRvaToSection	6D534B1F	16
ImageRvaToVa	6D52278A	17
ImagehlpApiVersion	6D51BE23	18
ImagehlpApiVersionEx	6D51BE29	19
MakeSureDirectoryPathExists	6D51BCDC	20

Line 15 of 99

■ 布局

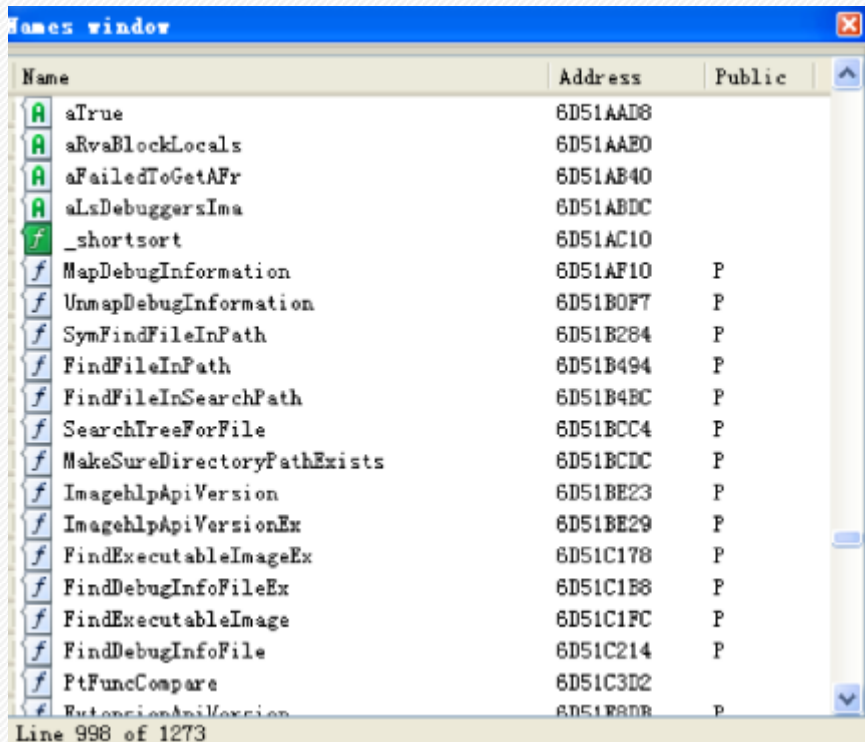
- 左侧是地址
- 中间是引用到的函数的名字
- 右侧是函数所在的DLL

■ 使用

- 选中感兴趣的API双击跳过去
- 查看上下文，分析其实现
- 观察传入参数和返回值

3 反汇编导航

Names



The screenshot shows a window titled "Names window" with a table of symbols. The table has three columns: "Name", "Address", and "Public". The symbols listed include "aTrue", "aRvaBlockLocals", "aFailedToGetAFr", "aLsDebuggersIma", "_shortsort", "MapDebugInformation", "UnmapDebugInformation", "SymFindFileInPath", "FindFileInPath", "FindFileInSearchPath", "SearchTreeForFile", "MakeSureDirectoryPathExists", "ImagehlpApiVersion", "ImagehlpApiVersionEx", "FindExecutableImageEx", "FindDebugInfoFileEx", "FindExecutableImage", "FindDebugInfoFile", "PtFuncCompare", and "RtensionApiVersion". The status bar at the bottom indicates "Line 998 of 1273".

Name	Address	Public
aTrue	6D51AAD8	
aRvaBlockLocals	6D51AAE0	
aFailedToGetAFr	6D51AB40	
aLsDebuggersIma	6D51ABDC	
_shortsort	6D51AC10	
MapDebugInformation	6D51AF10	P
UnmapDebugInformation	6D51B0F7	P
SymFindFileInPath	6D51B284	P
FindFileInPath	6D51B494	P
FindFileInSearchPath	6D51B4BC	P
SearchTreeForFile	6D51BCC4	P
MakeSureDirectoryPathExists	6D51BCDC	P
ImagehlpApiVersion	6D51BE23	P
ImagehlpApiVersionEx	6D51BE29	P
FindExecutableImageEx	6D51C178	P
FindDebugInfoFileEx	6D51C1B8	P
FindExecutableImage	6D51C1FC	P
FindDebugInfoFile	6D51C214	P
PtFuncCompare	6D51C3D2	
RtensionApiVersion	6D51F8D8	P

Line 998 of 1273

■ 布局

- 左侧是函数名和全局变量名
- 中间是名字对应的地址
- 右侧是访问权限

■ 使用

- 查看重命名后的列表
- 通过名字查看对应的地址
- 根据地址查看对应的名字

3 反汇编导航

3.1 双击导航

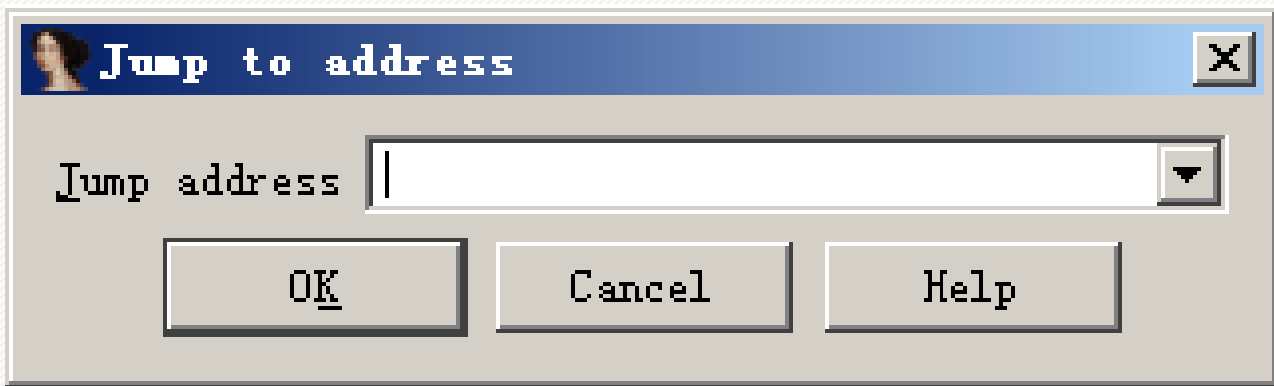
反汇编窗口中显示的任何名称都是导航目标，类似于网页中的超链接，只不过IDA中的名称需要双击才能访问。

```
.text:00401020 ; int __cdecl test(LPCSTR lpText, int)
.text:00401020 _test      proc near          ; CODE XREF: j__test↑j
.text:00401020
.text:00401020 var_40          = byte ptr -40h
.text:00401020 lpText          = dword ptr  8
.text:00401020 arg_4           = dword ptr  0Ch
.text:00401020
```

3 反汇编导航

3.2 跳转到地址

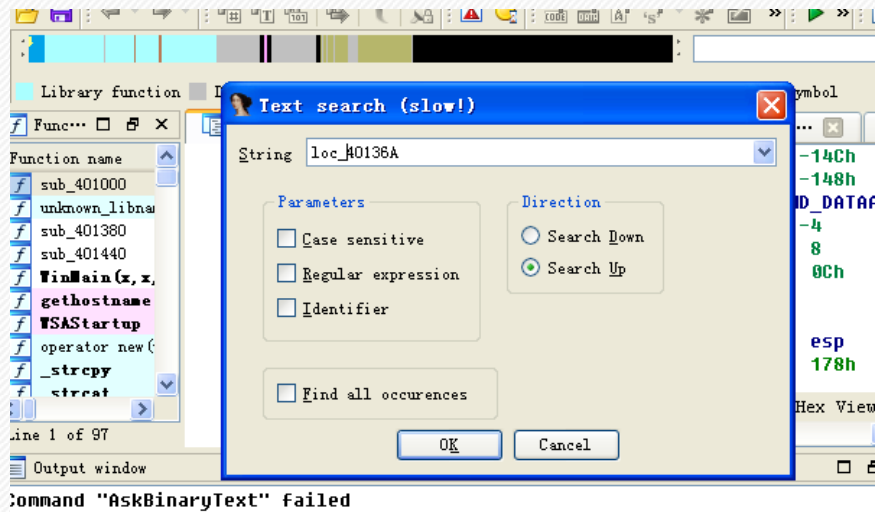
如果清楚导航的目的地址，我们可以用 Jump to Address对话框，可用G激活次对话框。



3 反汇编导航

3.3 搜索数据库 (文本搜索)

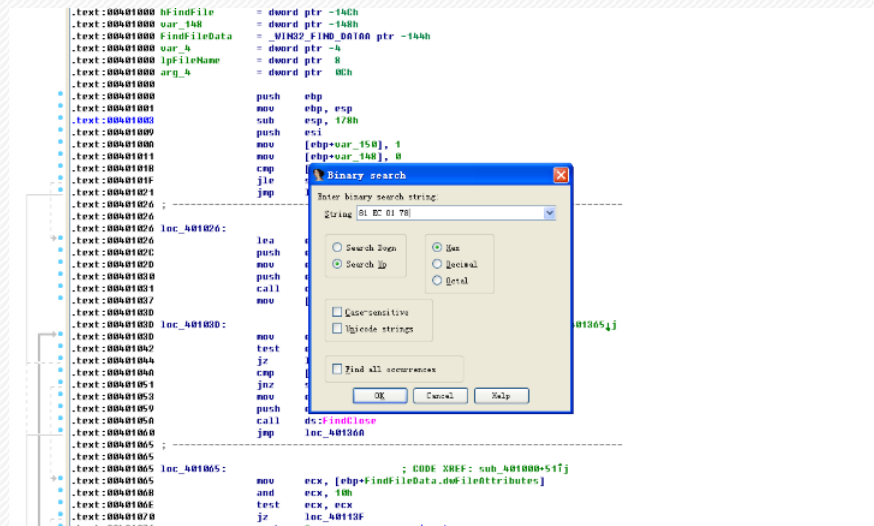
文本搜索相当于反汇编列表窗口进行字符串搜索。可通过ALT+T启动。



3 反汇编导航

3.3 搜索数据库（二进制搜索）

二进制搜索仅搜索十六进制视图窗口，可以搜索十六进制或ASCII字符串。可通过ALT+B



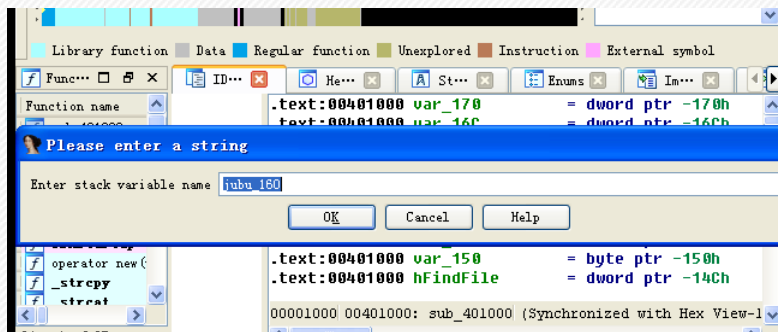
反汇编操作



4 反汇编操作

4.1 名称与命名

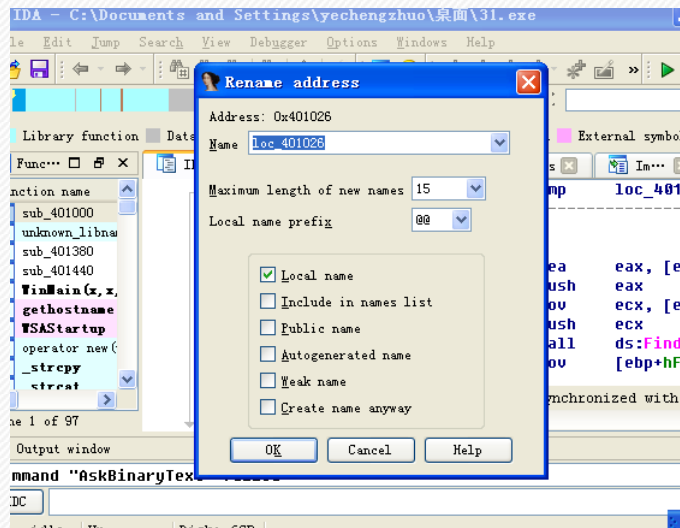
IDA允许随意修改任何名称，并处理在整个反汇编代码清单中扩散名称变更的所有细节。修改名称秩序单击希望修改的名称，并使用N打开更名对话框。



4 反汇编操作

4.1 名称与命名

重命名一个已命名的位置或给一个未命名的位置取名，这个过程与修改栈变量略有不同。



4 反汇编操作

4.1 名称与命名

Local names (局部名称) : 局部名称的作用域仅限于当前函数 ;

Include in names list (包含在名称列表中) : 将有一个名称被添加到名称窗口中 ;

Public name (公共名称) : 公共名称通常是由二进制文件 (如共享库) 输出的名称 ;

Autogenerated name (自动生成的名称) : 这个这个特性不会对反汇编代码产生任何明显的影响 ;

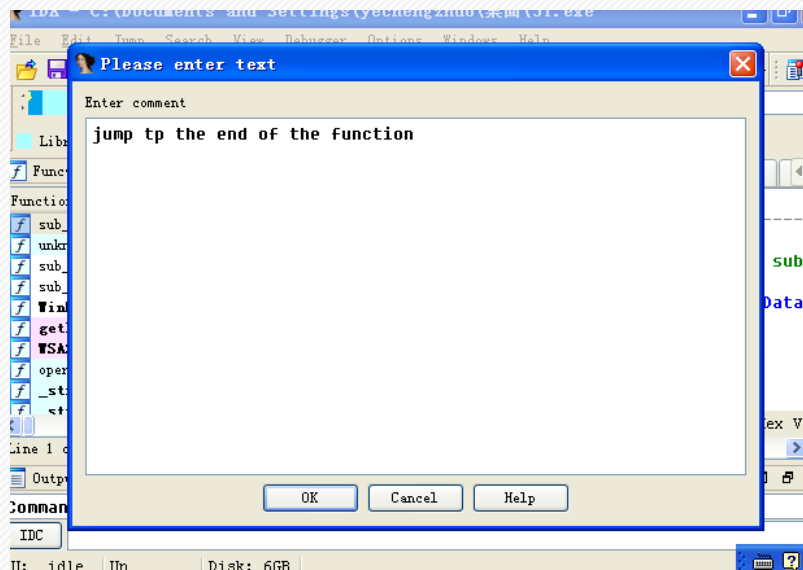
Weak name : 是公共符号的一种特殊形式 , 只是没有找到相同名称的公共符号来重写时才会用弱符号 ;

Create name anyway (无论如何都要创建名称) : 创建的名称的类型不同 , 行为也不一样。

4 反汇编操作

4.2 IDA中的注释

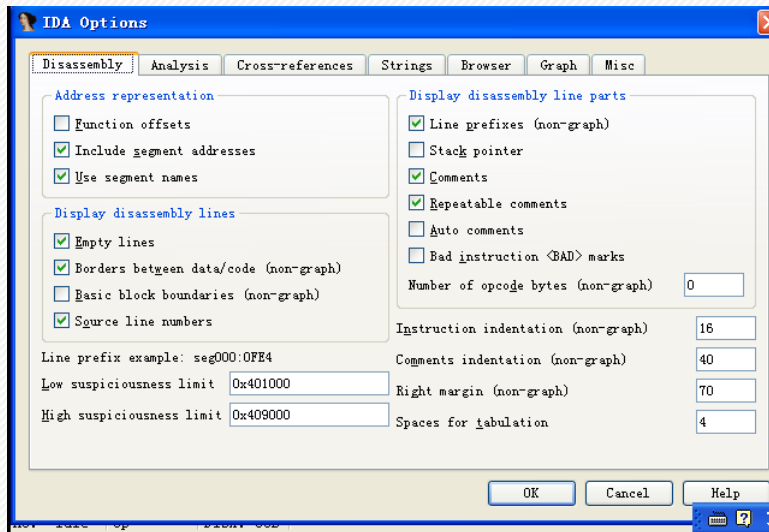
右击反汇编窗口右边缘，或者使用冒号（
: , ; ），可打开：输入注释对话框。
注意两者的区别



4 反汇编操作

4.3 基本代码转换（代码显示选项）

通过Options->General打开“IDA Options”对话框，并选择“disassembly”为每一个反汇编行选择其他需要显示的部分。



4.3 基本代码转换（代码显示选项）

Line prefixes（行前缀）：行前缀是每个反汇编行的section：address部分；

Stack pointer（栈指针）：IDA会详细分析每一个函数，以跟踪程序栈指针的变化；

Comments（注释）和Repeatable comments（可重复注释）：取消任何一个选项，IDA将不会显示对应类型的注释；

Auto comments（自动注释）：IDA可以为某些个指令类型自动添加注释；

Numbers of opcode bytes（操作码字节数）：IDA支持讲一个十六进制窗口与反汇编代码清单窗口同步。

4 反汇编操作

4.4 格式化指令操作数

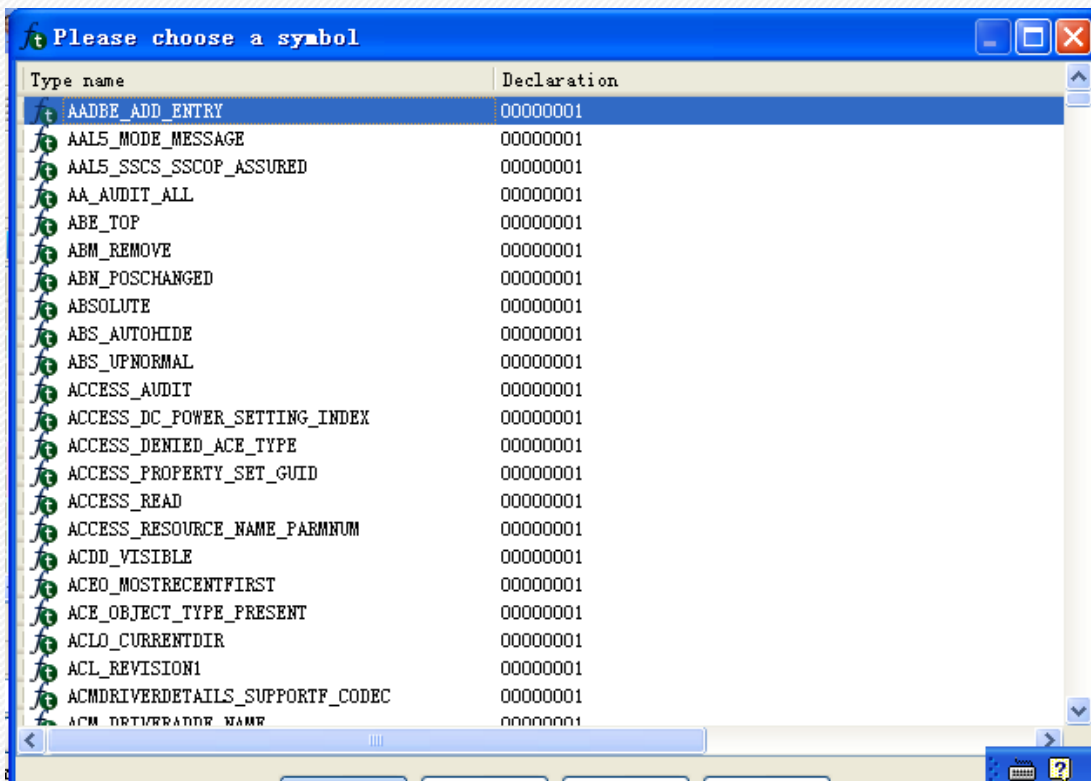


反汇编过程中，IDA会做出许多决定，确定如何格式化与每条指令有关的操作数。

比如以10进制来显示

4 反汇编操作

4.4 格式化指令操作数

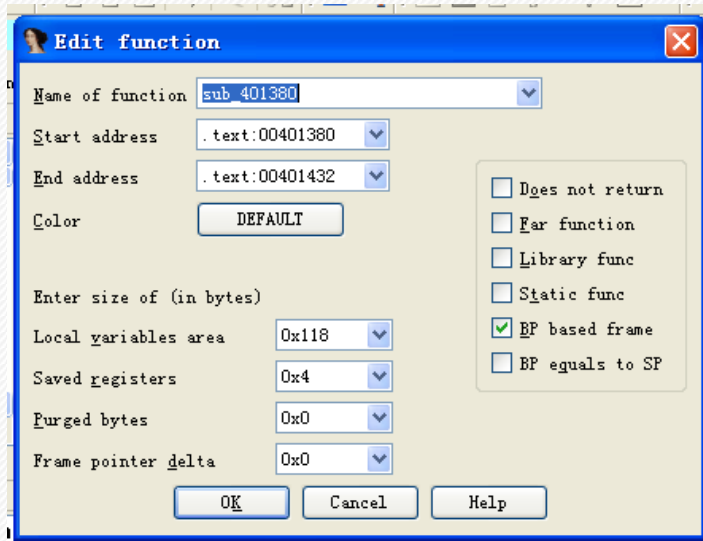


IDA维护着大量与许多常见库有关的已命名的常量，用户可以通过常量值的上下文菜单中的Use standard symbolic constant选项来访问这些常量。

4 反汇编操作

4.5 函数特性

IDA为它识别的每一个函数提供许多特性：函数名称、起始地址、结束地址、局部变量区、保存的寄存器、已删除字节等。



Name of function 为函数名称，提供另外一种更改函数名称的方法

Start address 为起始地址，函数中第一条指令的地址

通常，IDA 会在分析过程中，或根据创建函数时所使用的地址，自动识别这个地址

End address 为结束地址，函数中最后一条指令之后的地址

通常，它是函数的返回语句之后的指令的地址

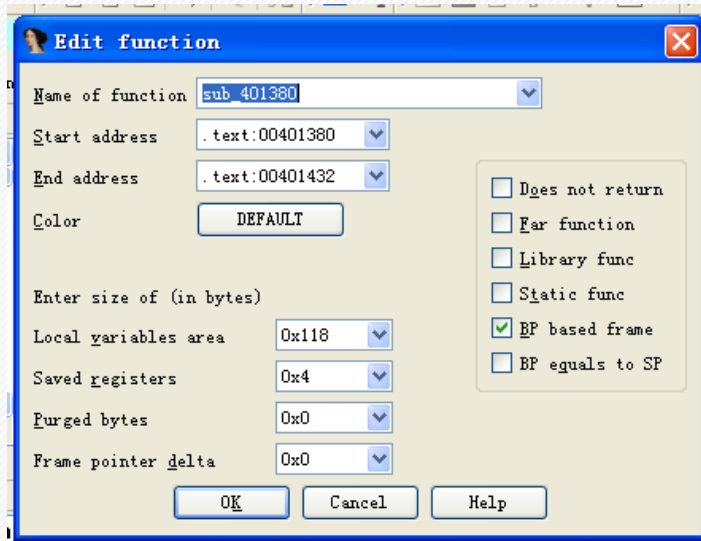
这个地址并不是函数的一部分，而是函数的最后一条指令之后的地址

Local variables area 为局部变量区，函数的局部变量（如下图）专用的栈字节数。

多数情况下，IDA 会通过分析函数的栈指针的行为，自动计算出这个值

4.5 函数特性

IDA为它识别的每一个函数提供许多特性：函数名称、起始地址、结束地址、局部变量区、保存的寄存器、已删除字节等。

**Save registers 为保存的寄存器**

为调用方保存寄存器（如上图）所使用的字节数，IDA认为保存的寄存器区域放在保存的返回地址顶部、与函数有关的所有局部变量的下方

一些编译器选择将寄存器保存在函数局部变量的顶部
IDA认为保存这些寄存器所使用的空间属于局部变量区域，而非保存的寄存器区域

Purged bytes 为已删除字节

已删除字节表示当函数返回调用方时，IDA从栈中删除的参数的字节数

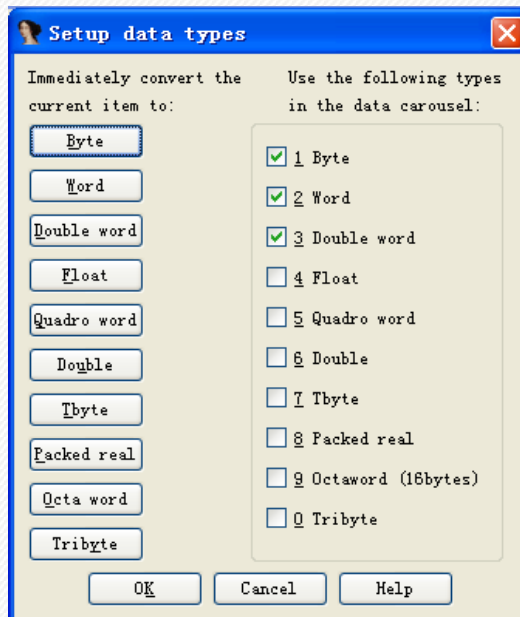
对 cdecl 函数而言，这个值始终为0

对 stdcall 函数来说，这个值表示传递到栈上的所有参数占用的空间

在 x86 程序中，如果 IDA 观察到程序使用了返回指令的 RET N 变体，它将自动确定这个值

4 反汇编操作

4.6 基本数据转换（指定数据大小）

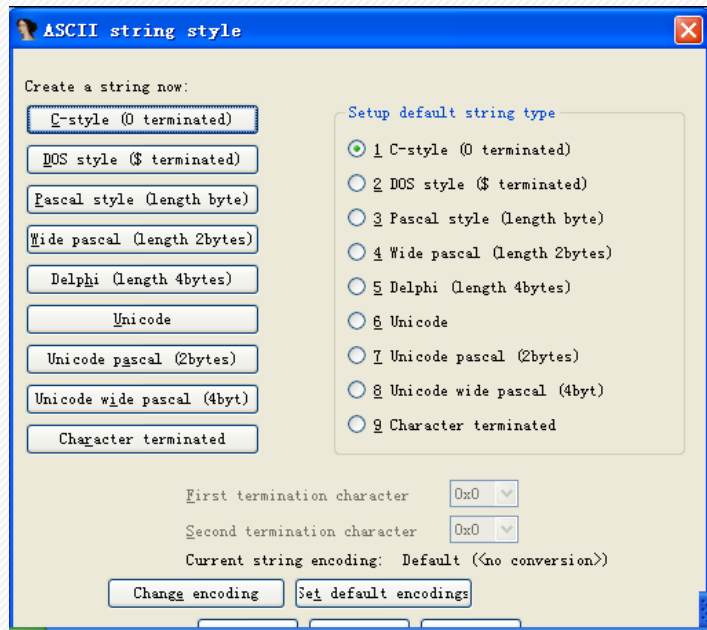


调整数据的大小是修改该数据最简单的办法。IDA提供了许多数据大小/类型说明符，最常见的说明符包括db、dw和dd。通过Options->Setup Data Types打开对话框。

4 反汇编操作

4.6 基本数据转换（处理字符串）

IDA能够识别大量字符串格式。默认情况下，IDA会搜索并格式化C风格、以空字符结尾的字符串。通过Options->ASCII String Style打开。



交叉引用



6.1 交叉引用

IDA中的交叉引用通常简称为xref。其中，有两类基本的交叉引用：代码交叉引用（CODE XREF）和数据交叉引用（DATA XREF）。

■ 代码交叉引用

- 那段代码调用了当前位置
- 那段代码会跳到当前位置

■ 数据交叉引用

- 那段代码会对该变量赋值
- 那段代码会读取该变量值

■ API交叉引用

- 重点强调基本API的意义。
- 多次封装，也离不开基本API

分析函数

■ 对函数的理解

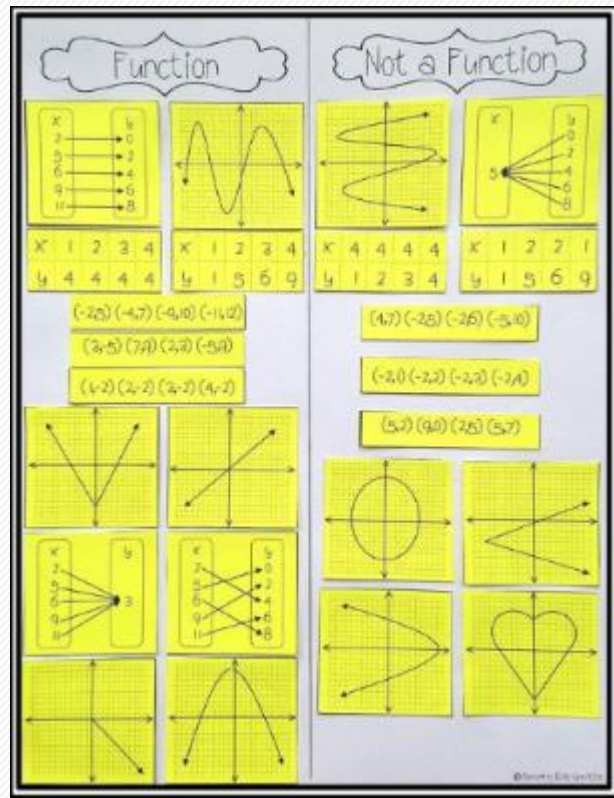
- 数学定义
- 编程定义

■ 可不可以一对多，多对多

- 可以
- 将对多内容封装，封装成结构体或者类

■ 识别要素

- 参数类型，参数值
- 返回类型，返回值
- 实现细节



重命名

■ 意义

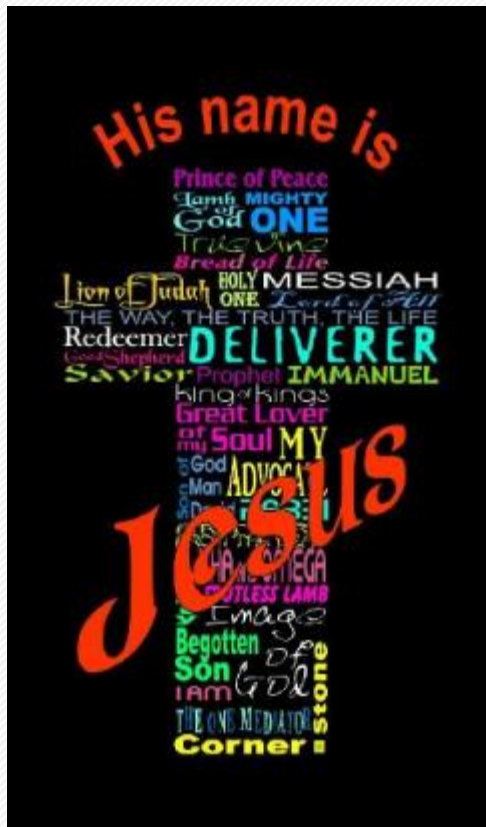
- 增加可读性，方便会议

■ 重命名对象

- 函数：函数名字、函数参数
- 变量：全局变量、局部变量，字符串，结构体，枚举

■ 记录

- IDA的本质是个笔记本
- 好记性不如烂笔头
- 形成自己的风格



6.1 交叉引用（代码交叉引用）

IDA中有两类交叉引用：

- 代码交叉引用 CODE XREF
 - 函数调用导致的交叉引用
 - 跳转交叉引用
- 数据交叉引用 DATA XREF
 - 读取交叉引用
 - 写入交叉引用
 - 偏移量交叉引用

6 交叉引用

6.1 交叉引用（代码交叉引用）

```
.text:00401000 sub_40100 proc near ; CODE XREF: main+2A ↓p
```

CODE XREF代表这是一个**代码交叉引用**，后面的地址（这里为_main+2A）是交叉引用的源头地址，即_main+2A处调用了该方法，交叉引用中使用的地址提供了额外的信息，指出交叉引用是在一个名为_main的函数中提出的，具体而言是_main函数中的第0x2A（42）字节。

地址后面总是有一个上行或下行箭头，表示引用位置的相对方向。需要向下滚动才能到达该地址。同样，上行箭头表示引用地址是一个较低的内存地址，需要向上滚动才能到达。

箭头后还包含了一个单字符后缀，代表交叉引用的类型，p代表这是一个调用类型，j则代表这是一个跳转类型的交叉引用。

6 交叉引用

6.1 交叉引用（数据交叉引用）

数据交叉引用用于跟踪二进制文件访问数据的方式。数据交叉引用与IDA数据库中任何牵涉到虚拟地址的字节有关（与数据交叉引用和栈变量无关）。IDA中有3种最常用的交叉引用分别用于表示某个位置何时被读取、何时被写入以及何时被引用。

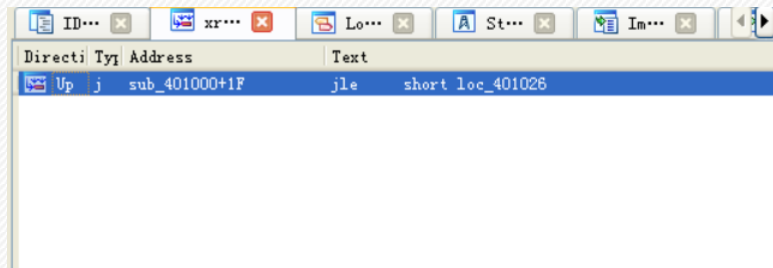
```
.data:0040B720 read_it      dd ?          ; DATA XREF: _main+E↑r
.data:0040B720             ; _main+25↑r
.data:0040B724 write_it    dd ?          ; DATA XREF: _main+1B↑w
.data:0040B724             ⑩; _main+2E↑w ...
.data:0040B728 ref_it      db ? ;      ; DATA XREF: _main+4↑o
.data:0040B729             db ? ;
.data:0040B72A             db ? ;
.data:0040B72B             db ? ;
```

读取交叉引用（read cross-reference）表示访问的是某个内存位置的内容，使用r作为单字符后缀，例如上面IDA反汇编中可以得知read_it被main+E处被读取。同理write_it处的写入交叉引用表示修改write_it处的值，使用w作为单字符后缀，值得注意的是，write_it位置显示的交叉引用以省略号处结束，表明对write_it的交叉引用数量超出了当前的交叉引用显示限制。你可以通过Options->General对话框中Cross-references选项卡中的Number of displayed xrefs（显示的交叉引用数量）设置修改这个限制。第三类数据交叉引用为偏移量交叉引用（offset cross-reference），它表示引用的是某个位置的地址（而非内容），使用o作为后缀。

6 交叉引用

6.2 交叉引用列表

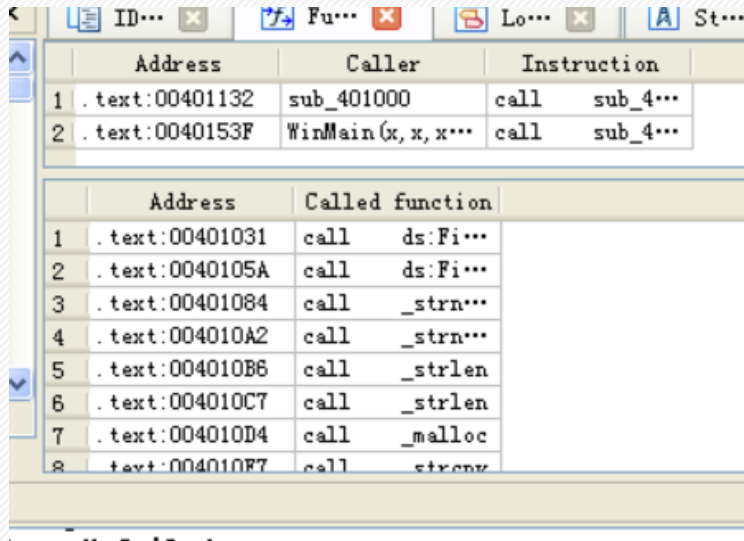
可以将光标放在一个或多个交叉引用的目标地址上，并选择View->Open SubViews->Cross-References打开。



6 交叉引用

6.2 函数调用

有一种交叉引用列表专门处理函数调用，通过View->Open SubViews->Function Calls打开。窗口的下半部分列出了当前函数做出的全部调用。



The screenshot shows a debugger window with the 'Function Calls' view selected. It contains two tables. The top table lists calls to 'sub_401000' from two different addresses. The bottom table lists various system and library functions called by the current function, such as 'ds:Fi...', '_strn...', '_strlen', and '_malloc'.

	Address	Caller	Instruction
1	.text:00401132	sub_401000	call sub_4...
2	.text:0040153F	WinMain(x, x, x...	call sub_4...

	Address	Called function
1	.text:00401031	call ds:Fi...
2	.text:0040105A	call ds:Fi...
3	.text:00401084	call _strn...
4	.text:004010A2	call _strn...
5	.text:004010B6	call _strlen
6	.text:004010C7	call _strlen
7	.text:004010D4	call _malloc
8	.text:004010E7	call strencv

```
#include <stdio.h>
#include <windows.h>

int test(char *szStr, int nNum)
{
    printf("%s, %d \r\n",szStr,nNum);
    MessageBox(NULL,szStr,NULL,MB_OK);

    return 5;
}

int main(int argc,char **argv)
{
    int nNum = test("hello",6);

    printf("%d \r\n",nNum);

    return 0;
}
```

在VC6下以DEBUG模式编译

C语言--逆向实例

IDA View-A	Hex View-1	Enums	Imports	Exports
Name	Address	Ordinal		
_mainCRTStartup	004011D0	[main entry]		

C语言--逆向实例

```
.text:004011F3      mov     [ebp+ms_exc.old_esp], esp
.text:004011F6      call    ds:__imp_GetVersion@0 ; GetVersion()
.text:004011FC      mov     __osver, eax
```

```
.text:0040120B      call    __IOHLL
.text:00401270      call    ds:__imp_GetCommandLineA@0 ; GetCommandLineA()
.text:00401276      mov     acmdln, eax
```

```
.text:00401270      mov     __acmdln, eax
.text:0040127B      call    __crtGetEnvironmentStringsA
.text:00401280      mov     __aenvptr, eax
.text:00401285      call    setargv
```

```
.text:0040129A      mov     __initenv, ecx
.text:004012A0      mov     edx, __environ
.text:004012A6      push    edx ; envp
.text:004012A7      mov     eax, __argv
.text:004012AC      push    eax ; argv
.text:004012AD      mov     ecx, __argc
.text:004012B3      push    ecx ; argc
.text:004012B4      call    _main_0
```

C语言--逆向实例

双击进入 main函数

```
.text:004010A0      push    ebp
.text:004010A1      mov     ebp, esp
.text:004010A3      sub     esp, 44h

.text:004010A6      push    ebx
.text:004010A7      push    esi
.text:004010A8      push    edi

.text:004010A9      lea     edi, [ebp+var_44]
.text:004010AC      mov     ecx, 11h
.text:004010B1      mov     eax, 0CCCCCCCCh
.text:004010B6      rep stosd
初始化为0xcc, 相当于int 3 , 方便调试
```

C语言--逆向实例

双击进入 main函数

```
.text:004010DD      pop     edi
.text:004010DE      pop     esi
.text:004010DF      pop     ebx
.text:004010E0      add     esp, 44h
.text:004010E3      cmp     ebp, esp
.text:004010E5      call    __chkesp
.text:004010EA      mov     esp, ebp
.text:004010EC      pop     ebp
.text:004010ED      retn
.text:004010ED _main      endp
.text:004010ED
```

C语言--逆向实例

双击进入 test函数

```
.text:00401020 _test      proc near      ; CODE XREF:
j__testj
.text:00401020
.text:00401020 var_40      = byte ptr -40h
.text:00401020 lpText      = dword ptr 8
.text:00401020 arg_4       = dword ptr 0Ch
.text:00401020
.text:00401020      push    ebp
.text:00401021      mov     ebp, esp
.text:00401023      sub     esp, 40h
.text:00401026      push    ebx
.text:00401027      push    esi
.text:00401028      push    edi
.text:00401029      lea     edi, [ebp+var_40]
.text:0040102C      mov     ecx, 10h
.text:00401031      mov     eax, 0CCCCCCCCh
.text:00401036      rep stosd
.text:00401038      mov     eax, [ebp+arg_4]
.text:0040103B      push    eax
.text:0040103C      mov     ecx, [ebp+lpText]
.text:0040103F      push    ecx
.text:00401040      push    offset Format ; "%s, %d
\r\n"
```

C语言--逆向实例

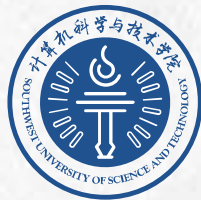
双击进入 test函数

.text:00401038
.text:0040103B
.text:0040103C
.text:0040103F
.text:00401040
.text:00401045
.text:0040104A
.text:0040104D
.text:0040104F
.text:00401051
.text:00401053
.text:00401056
.text:00401057
.text:00401059
.text:0040105F
为什么这里没有？

```
mov     eax, [ebp+arg_4]
push    eax
mov     ecx, [ebp+lpText]
push    ecx
push    offset Format ; "%s, %d \r\n"
call    _printf
add     esp, 0Ch;栈平衡
mov     esi, esp
push    0             ; uType
push    0             ; lpCaption
mov     edx, [ebp+lpText]
push    edx           ; lpText
push    0             ; hWnd
call    ds:__imp__MessageBoxA@16 ; MessageBoxA(x,x,x,x)
cmp     esi, esp
```

本次实验内容

- 1.按照实验指导书的内容，练习IDA PRO的使用。
- 2.完成35.exe的逆向，并给出逆向后的代码。（ C或者python均可）



计算机科学与技术学院
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

谢谢